What I learned building this

Notes to myself, written at the end of the water-potability project.

---

1. Check what a dummy scores before trusting any metric

The costly error here is unsafe water labelled safe, so recall on class 0
seemed like the obvious target. Then a dummy that predicts "not potable"
for everything scored $\text{recall}_0 = 1.0000$ — a perfect score for a model that
identifies zero safe samples.

Accuracy had the same problem in reverse: 61% of samples are one class, so
predicting that class always scores 61%.

Macro F1 was the first metric where nothing degenerate could win, because
it can't score well without doing something useful on both classes.

The habit: before trusting a metric, ask what the laziest possible model scores on it. If the answer is "well," the metric is measuring the class balance, not the model.

---

2. Recall$_0$ is the number that matters. It is not the metric to optimise.

These are different jobs and I conflated them at first.

- recall$_0$ = 0.8150 is the number with real-world cost — 18% of unsafe samples get called safe.
- Macro F1 is what selection runs on, precisely *because* optimising recall$_0$ directly has a degenerate solution.

A metric you report and a metric you optimise are not required to be the same metric.

---

3. Leakage is an ordering bug, and the mechanism is not memorisation

Computing medians before the train/test split means a statistic derived
from the test set gets written into the training data. The model never
sees test rows — what breaks is that the test set stops being an unbiased
estimate of unseen data.

Saying "the model memorises the test set" is wrong and invites questions I
can't defend. The correct framing: the score is inflated without the model
improving.

The fix was moving one line.

---

4. A plausible explanation is not a finding until it's tested

I made this mistake twice in the same README.

- Claimed a 0.040 gap between Kaggle and local came from an xgboost
  version difference. Never controlled for anything else.
- Claimed leakage was worth +0.0513, from a single observation at one seed.

Both sounded right. Both survived several revisions because they sounded
right. Both were wrong, and the controlled runs that disproved them took
under an hour combined.

**If I can't name the experiment that would falsify a claim, the claim
isn't ready to write down.**

---

5. One split is not a measurement

Across split seeds 0–9, clean macro F1 ranged 0.5826 to 0.6287 — mean
0.6068, std 0.0168.

The number I had been reporting, 0.5613, sits below all ten. Seed 42
produced an unusually hard test set, and I had been treating its output as
the score for weeks.

Any comparison smaller than the noise floor is not a result. With a
spread of 0.0168, a model A vs model B gap of 0.003 means nothing.

---

6. k-fold spread understates uncertainty

CV gave ± 0.0060. Resampling the split entirely gave ± 0.0168 — almost
three times wider.

All five folds draw from the same 2,620-row pool and share most of their
training data, so their agreement is partly an artifact of that sharing.

k-fold answers *"how consistent is this across folds of one sample."*
It does not answer *"how would this do on a different sample."* I needed
the second question and was reading the first one's answer.

---

7. Don't pick the seed that flatters the model

The deployed model scores below every seed I tested. Retraining on a
better seed would raise the reported number honestly-looking-ly — and it
would be selecting on the test set, the same error as

leakage in different
clothes.

The number stays. The distribution goes next to it.

---

8. Fixing a bug isn't always about the score

Leakage here didn't inflate the mean (−0.0092 across seeds, straddling
zero). It inflated the *spread*: 0.0265 leaky vs 0.0168 clean.

So the reason to fix it wasn't that it cheated — the medians were nearly
identical either way, so there was nothing to cheat with. The reason is
that a leaky pipeline returns a number unreliable in both directions.

A bug can be worth fixing for what it does to variance, not to the mean.

---

9. Errors surface downstream of the mistake

Two instances:

- An omitted optional field made pandas infer `object` dtype; the error appeared three layers into XGBoost, not at the fillna.
- A missing closing paren was reported by Python at the *next* line, since the parser only fails when it hits a token that can't continue the expression.

Where an error is reported and where it was caused are different places.

---

10. Reproducibility means recording the environment, not just the number

"macro F1 0.5613" isn't a claim. "macro F1 0.5613, xgboost 3.4.1, hyperparameters pinned, split seed 42" is one.

Practical consequences: pin hyperparameters instead of inheriting defaults, save with `save_model` rather than pickle, expose the running library version at `/ping` so a deployed instance can be checked against the environment its metric came from.

11. Weak data changes what the project is about

No feature correlates with the target above 0.05. A shuffled-label control
showed only weak signal. A 0.08% perturbation in two columns swings macro
F1 by up to five points.

On data like this, "which model wins" stops being answerable and the real
question becomes "can I measure a difference at all." Most of what I
built exists to answer the second question.

That's not a defect of the project. It's the reason the project taught me
anything.

---

12. On using AI

I used AI throughout — for structure, for scaffolding, for quizzing
myself. What that cost me, concretely:

- A restructured README shipped with `uvicorn app.main:app` when my

FastAPI object is named `api`. Anyone cloning it hit an instant crash.

- A response example was pasted under the wrong request, contradicting the paragraph two lines below it.
- A literal `0.xxxxxx` placeholder went into a published README.
- The reasoning sections got compressed into summary, because summary reads tighter — which deleted the dummy-classifier discovery, the one part of the project that was entirely mine.

Rule I'm keeping: AI edits prose, never facts. Commands, numbers, and example outputs get pasted from my terminal. Anything explaining *why* I did something, I write myself first.

And: don't keep code in the repo I couldn't rewrite from scratch.